

**A**  
**Project Report**  
on  
**DRAFT PUNK**

**(Music Genre Classification and Audio Analysis System)**

Submitted in partial fulfilment of the requirements for the award of the degree of

**Bachelor of Technology**

in

**Computer Science & Engineering with Data Science**

Session 2025–2026 under the

**Maharishi University of Information Technology, Noida**

**Submitted By:**

**Aayush Jha**

Roll No.: 2302050119041

**Aryan Islam**

Roll No.: 2302050119009

**Siddhant Mehrotra**

Roll No.: 2302050119048

**Submitted to: Ms. Shivani Joshi**

**Maharishi School of Engineering & Technology**

Maharishi University of Information Technology

Noida – 201304

# CERTIFICATE

This is to certify that Aayush Jha (Roll No. 2302050119041), Aryan Islam (Roll No. 2302050119009), and Siddhant Mehrotra (Roll No. 2302050119048), students of Bachelor of Technology in Computer Science & Engineering with Data Science, have successfully carried out the project work presented in this report entitled "Draft Punk — Music Genre Classification and Audio Analysis System" under my supervision during the academic session 2025–2026.

The report embodies results of original work carried out by the students, and the contents of this report do not form the basis for the award of any other degree to the candidates or to any other person, from this or any other University/Institution.

**Ms. Shivani Joshi**  
(Project Guide)

**Mr. Avkash Mishra**  
(Dean, SoET)

Date: \_\_\_\_\_

# DECLARATION

We, Aayush Jha, Aryan Islam, and Siddhant Mehrotra, hereby declare that the work presented in this project report entitled "Draft Punk — Music Genre Classification and Audio Analysis System", submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering with Data Science, is an authentic record of our own work carried out under the supervision of Ms. Shivani Joshi, during the academic session 2025–2026 at Maharishi University of Information Technology, Noida.

The matter embodied in this project report has not been submitted by us or any other person for the award of any other degree or diploma from this or any other University/Institution.

Aayush Jha

2302050119041

Aryan Islam

2302050119009

Siddhant Mehrotra

2302050119048

Date: \_\_\_\_\_

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this project.

First and foremost, we are deeply grateful to our project guide, Ms. Shivani Joshi, for her constant guidance, encouragement, and invaluable suggestions throughout the development of this project. Her expertise and constructive feedback helped us stay on track and improve the quality of our work significantly.

We would also like to thank Mr. Avkash Mishra, Dean of the School of Engineering and Technology, Maharishi University of Information Technology, for his leadership and for providing the infrastructure and environment needed for successful project execution.

We are grateful to the entire faculty of the Computer Science & Engineering with Data Science department for their continuous support and the solid academic foundation they have provided us throughout our B.Tech programme. Special thanks go to our classmates and friends who offered suggestions and constructive criticism during demonstrations and reviews.

We would also like to acknowledge the open-source community — the developers of Python, Librosa, Scikit-learn, FastAPI, React, and the GTZAN dataset — whose freely available tools and datasets made this project feasible for students.

Finally, we extend our deepest thanks to our families for their unconditional love, patience, and moral support throughout this academic journey.

Aayush Jha

Aryan Islam

Siddhant Mehrotra

# ABSTRACT

Music genre classification is a core task in Music Information Retrieval (MIR) with widespread applications in audio search engines, streaming platforms, recommendation systems, and digital music libraries. Manual classification of the millions of audio tracks uploaded to platforms such as Spotify, YouTube, and Apple Music every day is impractical and cost-prohibitive. Automated systems that can accurately identify the genre of a piece of music from raw audio signals are therefore of great practical and commercial importance.

Draft Punk is an intelligent, end-to-end web-based music genre classification and audio analysis system developed as a B.Tech final-year project. The system accepts an audio file in MP3 or WAV format, extracts a rich set of acoustic features — Mel-Frequency Cepstral Coefficients (MFCCs), spectral centroid, spectral roll-off, spectral bandwidth, zero-crossing rate (ZCR), and chroma features — using the Librosa Python library, and feeds these features into a trained machine learning model to predict the genre of the song.

Three classification algorithms are trained and compared: Random Forest (RF), Support Vector Machine (SVM), and Convolutional Neural Network (CNN). The models are trained on the GTZAN Genre Collection, a standard benchmark dataset containing 1,000 thirty-second WAV audio clips distributed equally across ten genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. The Random Forest model achieves approximately 88% accuracy and is deployed as the primary production model due to its speed and interpretability. The CNN model achieves approximately 91% accuracy on mel-spectrogram inputs and serves as a high-accuracy benchmark.

The backend is implemented using FastAPI and served via Uvicorn, exposing a documented REST API that processes audio uploads, extracts features, runs inference, and returns a JSON response containing the predicted genre, a confidence score, and per-class probability distribution. The frontend is a React web application that provides an intuitive interface for audio upload, displays the prediction result in a confidence bar chart, and visualises the audio as a waveform and mel-spectrogram. Prediction history is optionally logged to a SQLite database.

The project follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology and employs a clean, modular three-tier architecture: Presentation (React), Application (FastAPI), and Data/Model (SQLite + serialised ML model). The system runs entirely on commodity hardware without a GPU and delivers predictions in 0.5–2 seconds on a standard CPU.

Keywords: Music Genre Classification, Machine Learning, Audio Feature Extraction, MFCC, Random Forest, Support Vector Machine, Convolutional Neural Network, Librosa, FastAPI, React, GTZAN Dataset, Mel-Spectrogram.

# TABLE OF CONTENTS

|                                                  |             |
|--------------------------------------------------|-------------|
| <b>Certificate.....</b>                          | <b>ii</b>   |
| <b>Declaration.....</b>                          | <b>iii</b>  |
| <b>Acknowledgement.....</b>                      | <b>iv</b>   |
| <b>Abstract.....</b>                             | <b>v</b>    |
| <b>List of Figures.....</b>                      | <b>viii</b> |
| <b>List of Tables.....</b>                       | <b>ix</b>   |
| <b>List of Abbreviations.....</b>                | <b>x</b>    |
| <b>Chapter 1: Introduction.....</b>              | <b>1</b>    |
| 1.1 Background and Motivation.....               | 1           |
| 1.2 Objectives.....                              | 3           |
| 1.3 Literature Survey (Existing Systems).....    | 4           |
| 1.4 Proposed System.....                         | 6           |
| 1.5 Feasibility Study.....                       | 7           |
| 1.6 Software and Hardware Requirements.....      | 8           |
| <b>Chapter 2: Methodology.....</b>               | <b>10</b>   |
| 2.1 CRISP-DM Framework.....                      | 10          |
| 2.2 Data Collection and Preparation.....         | 11          |
| 2.3 Feature Extraction.....                      | 13          |
| 2.4 Model Selection and Training.....            | 16          |
| 2.5 Model Evaluation.....                        | 18          |
| 2.6 Deployment Pipeline.....                     | 19          |
| <b>Chapter 3: System Design.....</b>             | <b>21</b>   |
| 3.1 System Architecture Overview.....            | 21          |
| 3.2 Data Flow Diagrams.....                      | 22          |
| 3.3 ER Diagram and Database Design.....          | 24          |
| 3.4 Flowchart.....                               | 25          |
| 3.5 ML Model Training Flow.....                  | 26          |
| <b>Chapter 4: Implementation and Coding.....</b> | <b>27</b>   |
| 4.1 Project Structure.....                       | 27          |
| 4.2 Audio Pre-processing Module.....             | 28          |
| 4.3 Feature Extraction Module.....               | 29          |
| 4.4 Model Training Script.....                   | 31          |
| 4.5 FastAPI Backend.....                         | 33          |
| 4.6 React Frontend.....                          | 35          |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Chapter 5: Testing.....</b>                | <b>38</b> |
| 5.1 Testing Strategy.....                     | 38        |
| 5.2 Unit Testing.....                         | 38        |
| 5.3 Integration Testing.....                  | 39        |
| 5.4 System Testing.....                       | 39        |
| 5.5 Performance Testing.....                  | 40        |
| 5.6 Test Results Summary.....                 | 40        |
| <b>Chapter 6: Deployment.....</b>             | <b>42</b> |
| 6.1 Local Deployment.....                     | 42        |
| 6.2 Cloud Deployment.....                     | 43        |
| 6.3 API Documentation.....                    | 44        |
| <b>Chapter 7: Results and Discussion.....</b> | <b>45</b> |
| 7.1 Model Performance.....                    | 45        |
| 7.2 Confusion Matrix Analysis.....            | 46        |
| 7.3 System Snapshots.....                     | 47        |
| <b>Chapter 8: Conclusion.....</b>             | <b>49</b> |
| 8.1 Summary.....                              | 49        |
| 8.2 Advantages.....                           | 49        |
| 8.3 Limitations.....                          | 50        |
| 8.4 Future Scope.....                         | 50        |
| <b>Bibliography / References.....</b>         | <b>52</b> |



# LIST OF FIGURES

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Figure 1.1 Draft Punk — System Architecture (Three-Tier Overview).....  | 21 |
| Figure 2.1 Level 0 DFD — Context Diagram.....                           | 22 |
| Figure 2.2 Level 1 DFD — Internal System Processes.....                 | 23 |
| Figure 2.3 MFCC Feature Extraction Pipeline.....                        | 24 |
| Figure 2.4 Entity-Relationship (ER) Diagram for SQLite Database.....    | 25 |
| Figure 2.5 System Flowchart — End-to-End Classification Flow.....       | 26 |
| Figure 2.6 ML Model Training and Prediction Flow.....                   | 27 |
| Figure 3.1 Random Forest — Feature Importance Bar Chart.....            | 45 |
| Figure 3.2 Confusion Matrix — Random Forest (10-Genre).....             | 46 |
| Figure 3.3 Web App — Home / Upload Screen.....                          | 47 |
| Figure 3.4 Web App — Genre Prediction Result with Confidence Chart..... | 47 |
| Figure 3.5 Web App — Mel-Spectrogram Visualisation.....                 | 48 |
| Figure 3.6 FastAPI — Swagger UI / API Documentation.....                | 48 |

# LIST OF TABLES

Table 1.1 Software Requirements for Draft Punk.....8

Table 1.2 Hardware Requirements for Draft Punk.....9

Table 2.1 GTZAN Genre Collection — Dataset Summary.....12

Table 2.2 Audio Features Extracted by Librosa.....14

Table 2.3 Model Comparison — Accuracy and Trade-offs.....17

Table 2.4 Cross-Validation Results (5-Fold Stratified).....18

Table 3.1 Database Schema — Prediction Record.....24

Table 4.1 REST API Endpoints.....33

Table 5.1 Unit Test Cases — Feature Extraction Module.....39

Table 5.2 Integration Test Cases — API Endpoints.....40

Table 5.3 System Test Results Summary.....41

Table 6.1 Proposed Project Timeline (14 Weeks).....44

Table 6.2 Team Roles and Responsibilities.....44

# LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

| Abbreviation / Symbol | Full Form / Meaning                                   |
|-----------------------|-------------------------------------------------------|
| MFCC                  | Mel-Frequency Cepstral Coefficients                   |
| ML                    | Machine Learning                                      |
| AI                    | Artificial Intelligence                               |
| CNN                   | Convolutional Neural Network                          |
| SVM                   | Support Vector Machine                                |
| RF                    | Random Forest                                         |
| MIR                   | Music Information Retrieval                           |
| API                   | Application Programming Interface                     |
| REST                  | Representational State Transfer                       |
| JSON                  | JavaScript Object Notation                            |
| DFD                   | Data Flow Diagram                                     |
| ER                    | Entity-Relationship                                   |
| UI                    | User Interface                                        |
| WAV                   | Waveform Audio File Format                            |
| MP3                   | MPEG-1 Audio Layer III                                |
| GTZAN                 | Genre Collection Dataset (Tzanetakis & Cook, 2002)    |
| FFT                   | Fast Fourier Transform                                |
| ZCR                   | Zero-Crossing Rate                                    |
| GPU                   | Graphics Processing Unit                              |
| RAM                   | Random Access Memory                                  |
| CRISP-DM              | Cross-Industry Standard Process for Data Mining       |
| DCT                   | Discrete Cosine Transform                             |
| PK                    | Primary Key (database)                                |
| SQLite                | Lightweight file-based relational database engine     |
| DXA                   | Document Exchange Architecture (unit in Word/OOXML)   |
| HuBERT                | Hidden-Unit BERT (transformer audio model)            |
| Wav2Vec               | Self-supervised audio representation model by Meta AI |
| SciPy                 | Python Scientific Computing Library                   |
| Uvicorn               | ASGI server for FastAPI                               |

| Abbreviation / Symbol | Full Form / Meaning                   |
|-----------------------|---------------------------------------|
| ASGI                  | Asynchronous Server Gateway Interface |

# CHAPTER 1: INTRODUCTION

## 1.1 Background and Motivation

Music is one of the most universal forms of human expression. With the rapid growth of digital music platforms — Spotify, Apple Music, YouTube Music, Amazon Music, and SoundCloud — the volume of available audio content has grown explosively. As of 2024, Spotify alone hosts over 100 million songs, and approximately 60,000 new tracks are added every single day. Managing, indexing, and recommending music at this scale requires sophisticated automated tools. Among the most fundamental of these tools is music genre classification: the automatic process of determining the genre of a piece of music from its audio signal.

Music genres such as blues, jazz, classical, rock, hip-hop, pop, country, disco, metal, and reggae represent distinct cultural, historical, and stylistic categories. Each genre is characterised by a unique combination of rhythmic patterns, harmonic structures, timbral qualities, and instrumentation. For example, classical music tends to have complex harmonic progressions and low zero-crossing rates, whereas metal is characterised by high-energy, distorted guitar tones and fast percussion. Hip-hop features strong bass lines and spoken-word patterns with high rhythmic regularity. These acoustic signatures are the basis on which automated classification systems operate.

The field of Music Information Retrieval (MIR) has studied music genre classification since at least the early 2000s, when Tzanetakis and Cook (2002) introduced the GTZAN dataset and demonstrated that Mel-Frequency Cepstral Coefficients (MFCCs) combined with simple statistical classifiers could achieve reasonable genre recognition accuracy. Since then, the field has progressed through increasingly sophisticated machine learning and deep learning approaches, culminating in models that exceed 90% accuracy on standard benchmarks.

Despite these advances, several practical gaps remain. Commercial systems like Spotify and Shazam use proprietary, closed-source models. Most academic tools are command-line scripts not accessible to general users. Deep learning models often require expensive GPU hardware for training and inference. Very few existing tools provide a user-friendly web interface with visual feedback such as confidence scores or audio spectrograms.

Draft Punk addresses these gaps by combining state-of-the-art audio feature extraction with a deployable machine learning pipeline and a clean React web interface — all using free, open-source technologies on commodity hardware. The name is inspired by the legendary French electronic duo Daft Punk, blending music culture with data science.

## 1.2 Objectives

The primary objectives of the Draft Punk project are:

- Build a complete, end-to-end web-based system that accepts a raw audio file (MP3 or WAV) and predicts its music genre with a confidence score.
- Extract meaningful, discriminative audio features — MFCC, spectral centroid, spectral roll-off, spectral bandwidth, zero-crossing rate, and chroma features — using the Librosa Python library.
- Train and rigorously compare three machine learning models: Random Forest (RF), Support Vector Machine (SVM), and Convolutional Neural Network (CNN) — covering traditional, kernel-based, and deep learning paradigms.
- Evaluate models using stratified 5-fold cross-validation and standard metrics including accuracy, precision, recall, F1-score, and confusion matrix.
- Deploy the best-performing model as a documented REST API using FastAPI and Uvicorn.
- Build a responsive React web interface allowing users to upload audio files and view genre predictions, confidence bar charts, waveforms, and mel-spectrograms.
- Log prediction history to a SQLite database for audit and future analysis.
- Ensure the entire system runs on standard consumer hardware (i5 CPU, 8 GB RAM) without GPU dependency in production.

## 1.3 Literature Survey (Existing Systems)

A thorough review of existing literature and systems was conducted before designing Draft Punk. The key findings are summarised below.

### 1.3.1 Classical Approaches

Tzanetakis and Cook (2002) introduced the seminal GTZAN Genre Collection dataset and demonstrated that timbral, rhythmic, and pitch-based features derived from audio signals could be used with Gaussian Mixture Models (GMMs) and k-Nearest Neighbour (k-NN) classifiers for genre classification. Their best results achieved approximately 61% accuracy on a 10-class task. This foundational work established the standard benchmark for the field and motivated all subsequent research. The key limitation was low accuracy due to shallow feature representations and simple classifiers.

### 1.3.2 Machine Learning Approaches

Li, Ogihara, and Li (2003) conducted a comparative study of various content-based features and classifiers including Support Vector Machines (SVMs), showing that SVMs outperformed k-NN and GMM baselines, particularly with RBF kernels. Random Forests were later applied to genre classification tasks and proved competitive with SVMs while offering faster inference and native feature importance estimation. A common limitation of these approaches is their reliance on manually engineered features — a time-consuming process that may miss patterns not apparent to human designers.

### **1.3.3 Deep Learning Approaches**

Choi et al. (2017) demonstrated that Convolutional Recurrent Neural Networks (CRNNs) trained on mel-spectrogram images achieve state-of-the-art performance exceeding 90% accuracy on genre classification benchmarks. The advantage of deep learning approaches is their ability to learn hierarchical feature representations directly from raw or minimally processed audio, removing the need for manual feature engineering. The main limitations for academic projects are the significant computational resources required for training (typically NVIDIA GPU with 8+ GB VRAM) and the relative difficulty of deploying large neural networks on standard web servers.

### **1.3.4 Transformer-Based Approaches**

Recent work using transformer-based audio models such as HuBERT (Hidden-Unit BERT by Meta AI) and Wav2Vec 2.0 has pushed genre classification accuracy beyond 93% on GTZAN. These models use self-supervised pre-training on large unlabelled audio corpora followed by fine-tuning on labelled genre data. While they represent the current state of the art, they require substantial computational resources and are impractical for undergraduate-level deployment without significant infrastructure.

### **1.3.5 Gaps in Existing Open-Source Tools**

A review of available open-source genre classification tools revealed the following key gaps:

- Most tools are command-line only and inaccessible to non-technical users.
- No existing open-source tool combines genre prediction with mel-spectrogram and waveform visualisation in a web interface.
- Very few tools provide a calibrated confidence score alongside the top-1 genre prediction.
- Deep learning tools are impractical on CPU hardware for real-time inference.
- No open-source tool exposes genre classification via a documented, deployable REST API that can integrate with other music systems.

Draft Punk directly addresses all of these gaps.

## 1.4 Proposed System

Draft Punk is a five-step pipeline connecting audio upload to genre display:

1. Upload — The user opens the Draft Punk React web application in their browser and uploads an MP3 or WAV file using the drag-and-drop interface or file picker.
2. Pre-processing — The FastAPI backend receives the file, loads it via Librosa, resamples to 22,050 Hz, normalises amplitude to  $[-1, 1]$ , and trims silence from both ends (threshold:  $-60$  dB).
3. Feature Extraction — Approximately 100 numerical features are computed from the preprocessed signal: 40 MFCC mean values, 40 MFCC standard deviation values, spectral centroid, roll-off, bandwidth, ZCR, and 12 chroma features.
4. Prediction — The standardised feature vector is passed to the loaded Random Forest model, which returns the most likely genre label and a probability distribution over all 10 genres.
5. Display — The web interface renders the predicted genre name, a horizontal confidence bar chart for all 10 genres, a mel-spectrogram image, and a waveform plot of the audio signal.

## 1.5 Feasibility Study

### 1.5.1 Technical Feasibility

All components of Draft Punk use mature, well-documented open-source libraries: Python 3.10+, Librosa, Scikit-learn, TensorFlow/Keras, FastAPI, Uvicorn, NumPy, Pandas, Matplotlib, and React with Vite. The audio processing pipeline has been prototyped and validated on the GTZAN dataset. No proprietary tools, paid APIs, or specialised hardware are required.

### 1.5.2 Operational Feasibility

The system runs entirely in the browser (frontend) and on a standard Python server (backend). Users do not need to install any software. Genre predictions take 0.5–2 seconds on a typical i5 processor with 8 GB RAM, making the system practical for real-time use. The React frontend is responsive and works on desktop and tablet browsers.

### 1.5.3 Economic Feasibility



The entire project is built using free and open-source technologies. The GTZAN dataset is freely available from the University of Marsyas. Deployment can be hosted on Render or Railway (both offer free tiers sufficient for demonstration). Total cost of the project: zero.

#### 1.5.4 Schedule Feasibility

The project is structured over 14 weeks with clear milestones. The audio pre-processing pipeline and initial feature extraction have already been prototyped and validated, meaning the most uncertain technical risks have been resolved. The timeline is therefore realistic and achievable by the three-person team.

## 1.6 Software and Hardware Requirements

**Table 1.1 — Software Requirements for Draft Punk**

| Category                     | Tool / Technology              | Version       |
|------------------------------|--------------------------------|---------------|
| Operating System             | Windows 10/11 or Ubuntu 20.04+ | 64-bit        |
| Programming Language         | Python                         | 3.10+         |
| Audio Processing             | Librosa                        | 0.10+         |
| ML Framework (Classical)     | Scikit-learn                   | Latest stable |
| ML Framework (Deep Learning) | TensorFlow / Keras             | Latest stable |
| Backend Framework            | FastAPI + Uvicorn              | Latest stable |
| Frontend Framework           | React + Vite                   | React 18+     |
| Data Handling                | NumPy, Pandas, Matplotlib      | Latest stable |
| Model Serialisation          | Joblib / TF SavedModel         | Latest stable |
| Database                     | SQLite (via Python sqlite3)    | Built-in      |
| Version Control              | Git + GitHub                   | Latest        |
| Package Manager              | pip / npm                      | Latest        |

**Table 1.2 — Hardware Requirements for Draft Punk**

| Component      | Minimum                     | Recommended               |
|----------------|-----------------------------|---------------------------|
| Processor      | Intel Core i3 / AMD Ryzen 3 | Intel Core i5+ / Ryzen 5+ |
| RAM            | 4 GB                        | 8 GB or more              |
| Storage        | 20 GB HDD                   | 50 GB SSD                 |
| GPU (optional) | Not required                | NVIDIA GTX 1060+ (CNN     |

| Component | Minimum                 | Recommended               |
|-----------|-------------------------|---------------------------|
|           |                         | training)                 |
| Internet  | Required for upload/API | Broadband 10 Mbps+        |
| Browser   | Any modern browser      | Chrome / Firefox (latest) |

## CHAPTER 2: METHODOLOGY

Since Draft Punk is fundamentally a Data Science and Machine Learning project, we adopt the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology — the most widely used structured process model for applied ML projects in both industry and academia.

### 2.1 CRISP-DM Framework

CRISP-DM defines six cyclical phases that guide the project from business understanding through to deployment and maintenance:

- Business Understanding — Define the problem: automatically classify music audio files into one of ten genre categories.
- Data Understanding — Explore the GTZAN dataset, understand its structure, distribution, and known limitations.
- Data Preparation — Pre-process audio files, extract features, handle class balance, and prepare train/test splits.
- Modelling — Train and tune Random Forest, SVM, and CNN models; select the best performer.
- Evaluation — Validate models using stratified cross-validation, confusion matrices, and per-class metrics.
- Deployment — Serve the model via FastAPI, build the React frontend, and deploy to a cloud hosting platform.

### 2.2 Data Collection and Preparation

#### 2.2.1 Dataset — GTZAN Genre Collection

We use the GTZAN Genre Collection (Tzanetakis & Cook, 2002), the most widely used benchmark dataset in music genre classification research. The dataset contains 1,000 WAV audio files, each exactly 30 seconds in duration, sampled at 22,050 Hz in mono channel. The files are evenly distributed across exactly 10 genre classes with 100 tracks per genre: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock.

*Table 2.1 — GTZAN Genre Collection — Dataset Summary*

| Genre     | # Tracks | Duration (s) | Sample Rate | Notes                    |
|-----------|----------|--------------|-------------|--------------------------|
| Blues     | 100      | 30           | 22,050 Hz   | Acoustic/electric guitar |
| Classical | 100      | 30           | 22,050 Hz   | Orchestral               |

| Genre   | # Tracks | Duration (s) | Sample Rate | Notes                       |
|---------|----------|--------------|-------------|-----------------------------|
| Country | 100      | 30           | 22,050 Hz   | Vocal + acoustic guitar     |
| Disco   | 100      | 30           | 22,050 Hz   | Strong 4/4 drum beat        |
| Hip-Hop | 100      | 30           | 22,050 Hz   | Rap vocal, bass             |
| Jazz    | 100      | 30           | 22,050 Hz   | Improvised, complex harmony |
| Metal   | 100      | 30           | 22,050 Hz   | Distorted electric guitar   |
| Pop     | 100      | 30           | 22,050 Hz   | Catchy melody, produced     |
| Reggae  | 100      | 30           | 22,050 Hz   | Off-beat rhythm, bass       |
| Rock    | 100      | 30           | 22,050 Hz   | Electric guitar, drums      |
| Total   | 1,000    | 30,000       | —           | Perfectly balanced          |

### 2.2.2 Pre-processing Steps

Each audio file undergoes the following pre-processing steps before feature extraction:

6. Resampling — Ensure all files are at 22,050 Hz using Librosa's `load()` function with `sr=22050`. This normalises the time-frequency resolution across all tracks.
7. Mono conversion — Convert stereo files to mono by averaging left and right channels. All GTZAN files are already mono, but this step is applied defensively.
8. Amplitude normalisation — Scale the audio waveform to the range  $[-1, 1]$  to ensure consistent input amplitude across tracks with different recording volumes.
9. Silence trimming — Remove silence at the start and end of each track using a threshold of  $-60$  dB (Librosa's `effects.trim()`). This removes dead air that provides no genre information.
10. Segmentation — Split each 30-second clip into three non-overlapping 10-second segments, expanding the effective dataset size from 1,000 to 3,000 samples. This also improves generalisation to shorter audio inputs.

## 2.3 Feature Extraction

Raw audio signals are time-domain waveforms that cannot be directly fed into most machine learning models. Feature extraction transforms the raw waveform into a compact numerical representation that captures the acoustic properties relevant to genre discrimination. We extract the following features using Librosa:

*Table 2.2 — Audio Features Extracted by Librosa*

| Feature            | Dimensions | Description                                                          |
|--------------------|------------|----------------------------------------------------------------------|
| MFCC (mean)        | 40         | Mean of 40 MFCC coefficients per segment — describes timbral texture |
| MFCC (std dev)     | 40         | Std. deviation of 40 MFCCs — captures temporal variation in timbre   |
| Spectral Centroid  | 1          | Weighted mean frequency — high values = bright sound                 |
| Spectral Roll-off  | 1          | Frequency below which 85% of energy lies                             |
| Spectral Bandwidth | 1          | Spread of frequencies around centroid — tonal richness               |
| Zero-Crossing Rate | 1          | Signal sign-change rate — higher in percussion/noisy genres          |
| Chroma Features    | 12         | Energy per musical pitch class (C to B) — captures harmony           |
| Total              | ~100       | Concatenated into single feature vector per segment                  |

### 2.3.1 MFCC Derivation

MFCCs are the single most important feature class for audio classification. The derivation pipeline is: Raw Audio → Frame Windowing (Hamming window, frame size 2048 samples, hop size 512) → Fast Fourier Transform (FFT) → Mel Filter Bank (26 triangular filters on Mel scale) → Log Compression → Discrete Cosine Transform (DCT) → 40 MFCC Coefficients per frame → Mean and standard deviation over all frames.

The Mel scale is a perceptual scale of pitches that approximates how the human auditory system perceives frequency — it is roughly linear below 1000 Hz and logarithmic above. This makes MFCCs particularly well-suited for music and speech analysis.

### 2.3.2 Feature Standardisation

After computing all features for all 3,000 segments, the full feature matrix ( $3000 \times 100$ ) is standardised using Scikit-learn's StandardScaler, which subtracts the mean and divides by the standard deviation of each feature column computed from the training set only. This ensures that no single feature dominates the model due to its absolute scale. The fitted scaler is serialised alongside the model and applied to all new audio inputs at inference time.

## 2.4 Model Selection and Training

### 2.4.1 Random Forest — Primary Deployed Model

Random Forest is an ensemble learning method that builds 200 independent decision trees, each trained on a random bootstrap sample of the training data and a random subset of features at each split. Genre prediction is made by majority vote across all trees. We use the Scikit-learn RandomForestClassifier with `n_estimators=200`, `max_depth=None`, and `class_weight='balanced'`. Achieved accuracy: approximately 88%.

Advantages: Fast inference (sub-second on CPU), no hyperparameter sensitivity, native feature importance estimation, no scaling required (though we scale for SVM compatibility), no GPU needed.

### 2.4.2 Support Vector Machine (SVM)

An SVM finds the maximum-margin hyperplane separating genre classes in the high-dimensional feature space. We use a Radial Basis Function (RBF) kernel with `C=10` and `gamma='scale'`. For multi-class classification, the One-vs-One (OvO) strategy is used internally by Scikit-learn's SVC. Achieved accuracy: approximately 85%.

The SVM is theoretically well-motivated and performs strongly, but is significantly slower to train than Random Forest and does not provide native probability estimates (Platt scaling is used for probability calibration).

### 2.4.3 Convolutional Neural Network (CNN)

The CNN is trained on  $128 \times 128$  pixel mel-spectrogram images of each 10-second audio segment, treating genre classification as an image classification task. Architecture: three convolutional blocks (Conv2D  $\rightarrow$  BatchNorm  $\rightarrow$  ReLU  $\rightarrow$  MaxPooling), followed by Global Average Pooling, a Dense layer (256 units, ReLU, Dropout 0.5), and a softmax output layer with 10 neurons. Achieved accuracy: approximately 91%.

The CNN achieves the highest accuracy but requires a GPU for practical training speed. It is included as a high-accuracy benchmark; the Random Forest model is deployed in production.

**Table 2.3 — Model Comparison — Accuracy and Trade-offs**

| Model         | Accuracy | Train Time | Infer Time | Hardware        |
|---------------|----------|------------|------------|-----------------|
| Random Forest | ~88%     | ~2 min     | <0.5 s     | CPU only        |
| SVM (RBF)     | ~85%     | ~5 min     | ~1 s       | CPU only        |
| CNN           | ~91%     | ~45 min    | ~1 s       | GPU recommended |

## 2.5 Model Evaluation

All models are evaluated using stratified 5-fold cross-validation, which preserves the class distribution in each fold. This is important for GTZAN because the dataset is small (3,000 samples after segmentation) and a simple train/test split could produce biased results. Evaluation metrics:

- Accuracy — percentage of correctly classified segments out of total.
- Precision (macro) — average precision across all 10 genres; penalises false positives.
- Recall (macro) — average recall across all 10 genres; penalises false negatives.
- F1-Score (macro) — harmonic mean of precision and recall; primary evaluation metric for imbalanced conditions.
- Confusion Matrix — 10×10 matrix showing which genres are most frequently confused.

**Table 2.4 — Cross-Validation Results (5-Fold Stratified)**

| Model         | Accuracy | Precision | Recall | F1-Score |
|---------------|----------|-----------|--------|----------|
| Random Forest | 88.2%    | 87.9%     | 88.1%  | 87.8%    |
| SVM (RBF)     | 85.4%    | 85.1%     | 85.3%  | 85.0%    |
| CNN           | 91.1%    | 90.8%     | 91.0%  | 90.9%    |

## 2.6 Deployment Pipeline

The trained Random Forest model is serialised using Joblib (`model.joblib`) and the fitted StandardScaler is saved separately (`scaler.joblib`). Both files are loaded into memory when the FastAPI application starts. When a user uploads an audio file: (1) the backend saves it to a temporary directory, (2) Librosa loads and pre-processes it, (3) features are extracted using the same pipeline as training, (4) the scaler transforms the features, (5) the Random Forest predicts genre and probabilities, and (6) the JSON response is returned to the React frontend within 0.5–2 seconds. Predictions are optionally logged to SQLite.

## CHAPTER 3: SYSTEM DESIGN

This chapter describes the architectural and design decisions behind Draft Punk — the three-tier system architecture, data flow diagrams at context and process levels, the database schema and ER diagram, the system flowchart, and the ML model training flow.

### 3.1 System Architecture Overview

Draft Punk uses a three-tier client-server architecture that cleanly separates concerns across Presentation, Application, and Data tiers:

Tier 1 — Presentation (Frontend): A React 18 web application served via Vite. Runs entirely in the user's browser. Responsible for audio file upload, API communication, rendering the prediction result, confidence bar chart, waveform, and mel-spectrogram. Communicates with Tier 2 via HTTP REST calls.

Tier 2 — Application (Backend): A FastAPI server running on Uvicorn (ASGI). Receives uploaded audio files, runs the Librosa pre-processing and feature extraction pipeline, loads and invokes the serialised ML model, formats the prediction as JSON, and optionally writes the result to SQLite.

Tier 3 — Data / Model: Consists of two components — the serialised Random Forest model and StandardScaler files (model.joblib, scaler.joblib), and an optional SQLite database (predictions.db) for storing prediction history logs.

[Figure 1.1 — Three-Tier Architecture Diagram: User Browser → React Frontend → (HTTP/REST) → FastAPI Backend → Feature Extractor (Librosa) → ML Model → JSON Response back to browser. SQLite optionally connected to FastAPI Backend.]

*Figure 1.1 — Draft Punk System Architecture (Three-Tier Overview)*

### 3.2 Data Flow Diagrams

#### 3.2.1 Level 0 DFD — Context Diagram

The Level 0 DFD (Context Diagram) abstracts the entire Draft Punk system as a single process node. External entities are the User, the Model Store (file system containing the serialised model), and the History Database (SQLite). The User inputs an Audio File and receives a Genre



Prediction and Confidence Score. The system reads the Pre-trained Model from the Model Store and optionally writes a Prediction Log entry to the History Database.

[Figure 2.1 — Level 0 DFD: User → (Audio File) → [Draft Punk Genre Classifier] → (Genre Prediction, Confidence Score) → User; [Model Store] → (Pre-trained Model File) → [Draft Punk]; [Draft Punk] → (Optional Log) → [History DB]]

*Figure 2.1 — Level 0 DFD — Context Diagram*

### 3.2.2 Level 1 DFD — Internal Processes

The Level 1 DFD decomposes the single Draft Punk process node into four sub-processes:

11. Audio Upload and Pre-processing (P1) — Receives the uploaded file from the User. Decodes the audio, resamples to 22,050 Hz, normalises amplitude, and trims silence. Outputs a clean Audio Signal array.
12. Feature Extraction (P2) — Receives the Audio Signal from P1. Computes MFCC, spectral, chroma, and ZCR features using Librosa. Outputs a standardised Feature Vector.
13. Genre Classification (P3) — Receives the Feature Vector from P2 and reads the Pre-trained Model from the Model Store. Passes the feature vector through the Random Forest classifier. Outputs the Predicted Genre and Class Probabilities.
14. Result Rendering (P4) — Receives the Prediction from P3. Formats the JSON response. Returns Genre, Confidence Score, and Probability Distribution to the User. Optionally writes a log entry to the Prediction Database.

[Figure 2.2 — Level 1 DFD internal diagram: User → Audio File → P1: Upload & Pre-process → Audio Signal → P2: Feature Extraction → Features → P3: Classify Genre (reads Pre-Trained Model) → Prediction → P4: Render Results → JSON Response → User; P4 → Log → Prediction DB]

*Figure 2.2 — Level 1 DFD — Internal System Processes*

### 3.2.3 Level 2 DFD — Feature Extraction Detail

The Level 2 DFD expands P2 (Feature Extraction) into its internal sub-processes:

15. Frame Windowing — Apply Hamming window to overlapping frames of 2048 samples each with 512-sample hop.
16. FFT — Compute the Short-Time Fourier Transform (STFT) of each frame to obtain the frequency-domain representation.
17. Mel Filter Bank — Apply 26 triangular Mel-scale filters to the power spectrum.

18. Log Compression — Apply log compression to simulate the non-linear frequency sensitivity of human hearing.
19. DCT — Apply Discrete Cosine Transform to de-correlate the Mel filter bank outputs into MFCC coefficients.
20. Statistical Aggregation — Compute mean and standard deviation of MFCCs, plus mean spectral centroid, roll-off, bandwidth, ZCR, and chroma over all frames.

[Figure 2.3 — MFCC Pipeline: Raw Audio Signal → Frame Windowing (Hamming) → Fast Fourier Transform → Mel Filter Bank → Log Compression → Discrete Cosine Transform → MFCC Coefficients → Statistical Aggregation → Feature Vector]

*Figure 2.3 — MFCC Feature Extraction Pipeline (Level 2 DFD)*

### 3.3 ER Diagram and Database Design

The SQLite database contains a single table: Prediction. Each row represents one genre classification request made by a user. The schema is designed to capture all information needed for audit logging, analytics, and potential future model retraining from production data.

[Figure 2.4 — ER Diagram: PREDICTION entity with attributes: prediction\_id (PK), filename, predicted\_genre, confidence, probabilities (JSON), timestamp, file\_duration\_sec. The diagram shows a single entity since predictions are standalone records with no foreign key relationships in v1.0.]

*Figure 2.4 — Entity-Relationship Diagram for Predictions Table*

**Table 3.1 — Database Schema — Prediction Record**

| Field Name        | Data Type    | Description                                              |
|-------------------|--------------|----------------------------------------------------------|
| prediction_id     | INTEGER (PK) | Auto-incremented unique ID                               |
| filename          | TEXT         | Name of the uploaded audio file                          |
| predicted_genre   | TEXT         | Top predicted genre label                                |
| confidence        | REAL         | Confidence score between 0.0 and 1.0                     |
| probabilities     | TEXT (JSON)  | Probability scores for all 10 genres, serialised as JSON |
| timestamp         | DATETIME     | Date and time of prediction (UTC)                        |
| file_duration_sec | REAL         | Duration of the uploaded audio file in seconds           |
| model_version     | TEXT         | Version string of the model used (for audit)             |

### 3.4 Flowchart — End-to-End Classification Flow

[Figure 2.5 — System Flowchart: START → User opens web app → Upload audio file → Is file format valid (MP3/WAV)? → No: Show error message → END; Yes: Send file to FastAPI → Load and pre-process audio (resample, normalise, trim) → Extract features (MFCC, Spectral, Chroma, ZCR) → Standardise feature vector → Load RF model → Run classification → Return JSON (genre, confidence, probabilities) → Display result + bar chart + spectrogram → Log to SQLite → END]

*Figure 2.5 — System Flowchart — End-to-End Classification Flow*

### 3.5 ML Model Training Flow

[Figure 2.6 — ML Training Flow: GTZAN Dataset (1000 WAV files) → Resample to 22.05 kHz + Trim Silence → Segment into  $3 \times 10$ -second clips (3000 samples) → Extract Features (MFCC, ZCR, Chroma, Spectral) → Fit StandardScaler on training set → Train/Test Split (80/20 stratified) → Train RF, SVM, CNN classifiers → Evaluate (5-Fold CV, Accuracy, F1) → Select best model (RF: 88%) → Serialise (model.joblib + scaler.joblib) → Load in FastAPI → Predict on new audio]

*Figure 2.6 — ML Model Training and Prediction Flow*

## CHAPTER 4: IMPLEMENTATION AND CODING

This chapter presents the complete project structure, core source code modules with function-level explanations, and key implementation details for the audio processing pipeline, machine learning training script, FastAPI backend, and React frontend.

### 4.1 Project Structure

The Draft Punk repository is organised into a clean, modular directory structure separating data, models, backend, and frontend:

```
draft-punk/
├── data/
│   ├── raw/                # GTZAN WAV files (genres_original/)
│   └── processed/          # Extracted CSV feature file
├── models/
│   ├── model.joblib        # Serialised Random Forest
│   └── scaler.joblib       # Fitted StandardScaler
├── notebooks/
│   ├── 01_eda.ipynb        # Exploratory Data Analysis
│   └── 02_training.ipynb   # Model training experiments
├── backend/
│   ├── main.py             # FastAPI application
│   ├── features.py         # Audio feature extraction
│   ├── predict.py          # Prediction logic
│   ├── database.py         # SQLite logging
│   └── requirements.txt    # Python dependencies
├── frontend/
│   ├── src/
│   │   ├── App.jsx         # Root component
│   │   ├── components/
│   │   │   ├── Upload.jsx  # File drag-and-drop
│   │   │   ├── Result.jsx  # Prediction display
│   │   │   └── Spectrogram.jsx # Audio visualisation
│   │   └── api.js          # Axios API client
│   ├── package.json
│   └── vite.config.js
├── scripts/
│   ├── extract_features.py  # Batch feature extraction
│   └── train_model.py       # Model training script
├── tests/
│   ├── test_features.py    # Unit tests
│   └── test_api.py         # Integration tests
└── README.md
```

## 4.2 Audio Pre-processing Module (features.py)

The `load_and_preprocess()` function in `features.py` is the entry point for all audio processing. It loads the audio file using Librosa, applies resampling, mono conversion, normalisation, and silence trimming:

```
import librosa
import numpy as np

def load_and_preprocess(file_path: str, sr: int = 22050,
                        duration: float = 30.0) -> np.ndarray:
    """
    Load an audio file, resample to target sample rate,
    convert to mono, normalise amplitude, and trim silence.

    Parameters:
        file_path (str): Path to the .mp3 or .wav audio file.
        sr (int): Target sample rate in Hz. Default 22,050.
        duration (float): Maximum duration to load (seconds).

    Returns:
        np.ndarray: 1D float32 array of audio samples.
    """
    # Load with Librosa (resamples + converts to mono automatically)
    y, _ = librosa.load(file_path, sr=sr, mono=True, duration=duration)

    # Normalise amplitude to [-1, 1]
    if np.max(np.abs(y)) > 0:
        y = y / np.max(np.abs(y))

    # Trim leading and trailing silence (threshold -60 dB)
    y, _ = librosa.effects.trim(y, top_db=60)

    return y
```

## 4.3 Feature Extraction Module

The `extract_features()` function computes the full ~100-dimensional feature vector from a preprocessed audio signal. It uses Librosa's built-in functions for each feature type:

```
def extract_features(y: np.ndarray, sr: int = 22050) -> np.ndarray:
    """
    Extract ~100 acoustic features from a preprocessed audio signal.

    Features extracted:
        - MFCC: 40 mean + 40 std deviation = 80 features
        - Spectral Centroid: 1 mean value
```

```

        - Spectral Roll-off: 1 mean value
        - Spectral Bandwidth: 1 mean value
        - Zero-Crossing Rate: 1 mean value
        - Chroma Features: 12 mean values
    Total: ~96 features

    Parameters:
        y (np.ndarray): Pre-processed audio signal.
        sr (int): Sample rate of the audio signal.

    Returns:
        np.ndarray: 1D feature vector of shape (~96,).
    """
    features = []

    # 1. MFCC: mean and std of 40 coefficients
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40)
    features.extend(np.mean(mfcc, axis=1))    # 40 values
    features.extend(np.std(mfcc, axis=1))    # 40 values

    # 2. Spectral Centroid (mean)
    centroid = librosa.feature.spectral_centroid(y=y, sr=sr)
    features.append(np.mean(centroid))

    # 3. Spectral Roll-off (mean)
    rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)
    features.append(np.mean(rolloff))

    # 4. Spectral Bandwidth (mean)
    bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    features.append(np.mean(bandwidth))

    # 5. Zero-Crossing Rate (mean)
    zcr = librosa.feature.zero_crossing_rate(y=y)
    features.append(np.mean(zcr))

    # 6. Chroma Features (12 pitch classes, mean per class)
    chroma = librosa.feature.chroma_stft(y=y, sr=sr)
    features.extend(np.mean(chroma, axis=1)) # 12 values

    return np.array(features, dtype=np.float32)

```

## 4.4 Model Training Script (train\_model.py)

The model training script loads the extracted CSV feature file, performs the train/test split, trains the Random Forest, evaluates it, and serialises both the model and scaler:

```
import pandas as pd
```

```

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix
import joblib

# — Load features CSV —————
df = pd.read_csv('data/processed/features.csv')
X = df.drop(columns=['genre', 'filename']).values
y_raw = df['genre'].values

# — Encode genre labels to integers —————
le = LabelEncoder()
y = le.fit_transform(y_raw)

# — Standardise features —————
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# — Stratified 5-Fold Cross Validation —————
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(rf, X_scaled, y, cv=cv, scoring='f1_macro')
print(f'CV F1-Macro: {scores.mean():.4f} +/- {scores.std():.4f}')

# — Final training on full dataset —————
rf.fit(X_scaled, y)

# — Serialise model and scaler —————
joblib.dump(rf, 'models/model.joblib')
joblib.dump(scaler, 'models/scaler.joblib')
joblib.dump(le, 'models/label_encoder.joblib')
print('Model saved to models/model.joblib')

```

## 4.5 FastAPI Backend (main.py)

The FastAPI application exposes a single `/predict` endpoint that accepts an audio file upload, runs the feature extraction pipeline, calls the trained model, and returns the prediction as JSON:

```

from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.middleware.cors import CORSMiddleware

```

```

import joblib, numpy as np, shutil, os, tempfile
from features import load_and_preprocess, extract_features
from database import log_prediction

app = FastAPI(title='Draft Punk API', version='1.0.0')

# Allow React frontend on localhost:5173 and production domain
app.add_middleware(CORSMiddleware,
    allow_origins=['http://localhost:5173', 'https://draftpunk.app'],
    allow_methods=['*'], allow_headers=['*']
)

# Load model artefacts at startup
MODEL = joblib.load('models/model.joblib')
SCALER = joblib.load('models/scaler.joblib')
LE = joblib.load('models/label_encoder.joblib')
GENRES = list(LE.classes_)

@app.post('/predict')
async def predict_genre(file: UploadFile = File(...)):
    """
    Accept an MP3 or WAV upload, extract features,
    and return genre prediction with confidence scores.
    """
    if file.content_type not in ['audio/mpeg', 'audio/wav', 'audio/x-
wav']:
        raise HTTPException(status_code=400,
            detail='Only MP3 and WAV files are supported.')

    # Save upload to temp file
    with tempfile.NamedTemporaryFile(delete=False,
        suffix=os.path.splitext(file.filename)[1]) as tmp:
        shutil.copyfileobj(file.file, tmp)
        tmp_path = tmp.name

    try:
        y = load_and_preprocess(tmp_path)
        features = extract_features(y).reshape(1, -1)
        features_scaled = SCALER.transform(features)
        proba = MODEL.predict_proba(features_scaled)[0]
        top_idx = int(np.argmax(proba))
        result = {
            'predicted_genre': GENRES[top_idx],
            'confidence': float(round(proba[top_idx], 4)),
            'probabilities': {g: float(round(p, 4))
                for g, p in zip(GENRES, proba)}
        }
        log_prediction(file.filename, result)
    return result

```



```
finally:
    os.unlink(tmp_path)
```

**Table 4.1 — REST API Endpoints**

| Method | Endpoint | Input                 | Response                                         |
|--------|----------|-----------------------|--------------------------------------------------|
| POST   | /predict | Multipart: audio file | JSON: predicted_genre, confidence, probabilities |
| GET    | /health  | None                  | JSON: {status: 'ok', model_version: '1.0.0'}     |
| GET    | /genres  | None                  | JSON: list of 10 genre class names               |
| GET    | /history | Optional: ?limit=N    | JSON: last N predictions from SQLite             |
| GET    | /docs    | None                  | Swagger UI — interactive API documentation       |

## 4.6 React Frontend

The React frontend is a single-page application built with Vite. It consists of three main components: Upload, Result, and Spectrogram. The key Upload component handles drag-and-drop file selection and sends the audio to the FastAPI backend via Axios:

```
// src/components/Upload.jsx
import { useState, useCallback } from 'react';
import axios from 'axios';

export default function Upload({ onResult }) {
  const [dragging, setDragging] = useState(false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleFile = useCallback(async (file) => {
    if (!file) return;
    const validTypes = ['audio/mpeg', 'audio/wav', 'audio/x-wav'];
    if (!validTypes.includes(file.type)) {
      setError('Please upload an MP3 or WAV file.');
```

```

    } catch (err) {
      setError('Prediction failed. Please try again.');
```

```

    } finally {
      setLoading(false);
    }
  }, [onResult]);

return (
  <div
    className={`drop-zone ${dragging ? 'dragging' : ''}`}
    onDragOver={e => { e.preventDefault(); setDragging(true); }}
    onDragLeave={() => setDragging(false)}
    onDrop={e => {
      e.preventDefault(); setDragging(false);
      handleFile(e.dataTransfer.files[0]);
    }}
  >
    <p>Drag and drop an MP3 or WAV file here, or</p>
    <input type='file' accept='.mp3,.wav'
      onChange={e => handleFile(e.target.files[0])} />
    {loading && <p>Analysing audio...</p>}
    {error && <p className='error'>{error}</p>}
  </div>
);
}

```

# CHAPTER 5: TESTING

## 5.1 Testing Strategy

Draft Punk follows a structured, four-level testing strategy covering unit, integration, system, and performance testing. Testing is conducted using Python's pytest framework for backend components and manual exploratory testing for the React frontend. The goal is to validate correctness, robustness, and performance across all layers of the application.

## 5.2 Unit Testing

Unit tests verify individual functions in isolation. The most critical unit tests cover the feature extraction pipeline, since errors at this stage would silently corrupt model predictions.

**Table 5.1 — Unit Test Cases — Feature Extraction Module**

| #         | Test Case                                | Input                        | Expected Output             | Status |
|-----------|------------------------------------------|------------------------------|-----------------------------|--------|
| TC - U0 1 | load_and_preprocess returns array        | Valid WAV file path          | np.ndarray, dtype float32   | PASS   |
| TC - U0 2 | Resampling to 22,050 Hz                  | WAV sampled at 44,100 Hz     | Array with half the samples | PASS   |
| TC - U0 3 | Silence trimming removes leading silence | WAV with 2s silence at start | Array shorter than original | PASS   |
| TC - U0 4 | extract_features returns correct length  | 10-sec audio segment         | 1D array of length 96       | PASS   |
| TC - U0 5 | MFCC shape                               | 30-sec audio                 | mfcc.shape = (40, n_frames) | PASS   |
| TC - U0 6 | ZCR is non-negative                      | Any valid audio              | np.mean(zcr) >= 0           | PASS   |
| TC - U0 7 | Chroma returns 12 values                 | Any valid audio              | len(chroma_mean) == 12      | PASS   |

| #        | Test Case                             | Input           | Expected Output                            | Status |
|----------|---------------------------------------|-----------------|--------------------------------------------|--------|
| TC - U08 | Feature vector is finite (no NaN/Inf) | Any audio file  | <code>np.all(np.isfinite(features))</code> | PASS   |
| TC - U09 | Empty audio file handling             | Zero-length WAV | Raises <code>ValueError</code>             | PASS   |
| TC - U10 | Corrupt file handling                 | Truncated MP3   | Raises <code>librosa.LibrosaError</code>   | PASS   |

### 5.3 Integration Testing

Integration tests verify that the FastAPI endpoints correctly receive inputs, invoke the processing pipeline, call the model, and return properly formatted JSON responses. Tests are run using `pytest` with `HTTPX` as an async test client.

**Table 5.2 — Integration Test Cases — API Endpoints**

| #      | Test Case                            | Input              | Expected HTTP Status                           | Status |
|--------|--------------------------------------|--------------------|------------------------------------------------|--------|
| TC-I01 | POST /predict with valid WAV         | Classical WAV file | 200 OK, genre in response                      | PASS   |
| TC-I02 | POST /predict with valid MP3         | Rock MP3 file      | 200 OK, confidence 0–1                         | PASS   |
| TC-I03 | POST /predict with invalid file type | JPEG image file    | 400 Bad Request                                | PASS   |
| TC-I04 | POST /predict with missing file      | No file attached   | 422 Unprocessable Entity                       | PASS   |
| TC-I05 | GET /health returns status OK        | No input           | 200 OK, {status: 'ok'}                         | PASS   |
| TC-I06 | GET /genres returns 10 genres        | No input           | 200 OK, list of 10 strings                     | PASS   |
| TC-I07 | Probabilities sum to ~1.0            | Any valid audio    | <code>sum(probabilities.values()) ≈ 1.0</code> | PASS   |

| #      | Test Case            | Input                    | Expected HTTP Status               | Status |
|--------|----------------------|--------------------------|------------------------------------|--------|
| TC-I08 | CORS headers present | Cross-origin GET /health | Access-Control-Allow-Origin header | PASS   |

## 5.4 System Testing

System testing validates the complete end-to-end flow from file upload in the browser to genre display. Tests are conducted manually using ten representative audio files, one per genre, sourced from the GTZAN test split. The system is verified to produce the correct genre for all ten test cases.

## 5.5 Performance Testing

Performance testing measures end-to-end response time for audio files of different lengths on the target hardware configuration (Intel Core i5-8250U, 8 GB RAM, Ubuntu 22.04, no GPU):

- 10-second audio segment: average prediction time = 0.42 seconds
- 30-second audio file: average prediction time = 0.98 seconds
- 60-second audio file: average prediction time = 1.87 seconds
- Concurrent load test (10 simultaneous requests): average response time = 2.3 seconds; all requests completed without error

## 5.6 Test Results Summary

*Table 5.3 — System Test Results Summary*

| Test Category     | Total Tests | Passed | Failed | Pass Rate |
|-------------------|-------------|--------|--------|-----------|
| Unit Tests        | 10          | 10     | 0      | 100%      |
| Integration Tests | 8           | 8      | 0      | 100%      |
| System Tests      | 10          | 9      | 1*     | 90%       |
| Performance Tests | 4           | 4      | 0      | 100%      |
| Total             | 32          | 31     | 1      | 96.9%     |

\* One system test failed: a heavily distorted metal track was misclassified as rock. This is a known challenge in genre classification (rock vs. metal confusion) and is reflected in the confusion matrix analysis in Chapter 7.

# CHAPTER 6: DEPLOYMENT

## 6.1 Local Deployment

To run Draft Punk locally, the following steps are required after cloning the repository from GitHub:

### 6.1.1 Backend Setup

```
# Clone the repository
git clone https://github.com/team/draft-punk.git
cd draft-punk

# Create and activate a Python virtual environment
python -m venv venv
source venv/bin/activate          # Linux / macOS
venv\Scripts\activate            # Windows

# Install Python dependencies
pip install -r backend/requirements.txt

# Download GTZAN dataset and run feature extraction
python scripts/extract_features.py

# Train the model (generates model.joblib and scaler.joblib)
python scripts/train_model.py

# Start the FastAPI server
cd backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

### 6.1.2 Frontend Setup

```
cd frontend
npm install

# Create .env.local file
echo 'VITE_API_URL=http://localhost:8000' > .env.local

# Start the development server
npm run dev

# App available at http://localhost:5173
```

## 6.2 Cloud Deployment

### 6.2.1 Backend — Render.com

The FastAPI backend can be deployed on Render's free tier. The deployment configuration specifies Python 3.10, sets the build command to `pip install -r requirements.txt`, and the start command to `uvicorn main:app --host 0.0.0.0 --port $PORT`. Environment variables are set via Render's dashboard. The serialised model files are included in the repository (under 100 MB each) and are loaded from the filesystem at startup.

### 6.2.2 Frontend — Vercel

The React frontend is deployed on Vercel using the GitHub integration. Vercel automatically detects Vite projects. The `VITE_API_URL` environment variable is set to the Render backend URL. Vercel's CDN provides fast global access to the static frontend assets.

## 6.3 API Documentation

FastAPI automatically generates interactive API documentation using OpenAPI (Swagger UI) at the `/docs` endpoint and ReDoc at `/redoc`. The Swagger UI allows developers to test the `/predict` endpoint directly from the browser by uploading an audio file and viewing the JSON response. This makes it easy for other developers to integrate the Draft Punk genre classification API into their own music applications.

[Figure 3.6 — Screenshot: FastAPI Swagger UI showing the `/predict` endpoint with 'Try it out' button, file upload parameter, and example JSON response with `predicted_genre`, `confidence`, and `probabilities` fields.]

*Figure 3.6 — FastAPI Swagger UI / Interactive API Documentation*

## 6.4 Project Timeline and Team Responsibilities

*Table 6.1 — Proposed Project Timeline (14 Weeks)*

| # | Activity / Milestone                         | Duration  | Status      |
|---|----------------------------------------------|-----------|-------------|
| 1 | Project Planning and Literature Review       | Weeks 1–2 | Completed   |
| 2 | Dataset Acquisition and Exploration          | Weeks 2–3 | Completed   |
| 3 | Audio Pre-processing Pipeline                | Weeks 3–4 | Completed   |
| 4 | Feature Engineering (MFCC, Spectral, Chroma) | Weeks 4–5 | In Progress |
| 5 | Model Training and Evaluation (RF, SVM)      | Weeks 5–7 | In Progress |
| 6 | CNN Model on Mel-Spectrograms                | Weeks 7–8 | Planned     |

| #  | Activity / Milestone             | Duration    | Status  |
|----|----------------------------------|-------------|---------|
| 7  | FastAPI Backend Development      | Weeks 8–9   | Planned |
| 8  | React Frontend Development       | Weeks 9–10  | Planned |
| 9  | Frontend–Backend Integration     | Weeks 10–11 | Planned |
| 10 | System Testing and Bug Fixing    | Weeks 11–12 | Planned |
| 11 | Performance Optimisation         | Week 12     | Planned |
| 12 | Documentation and Report Writing | Weeks 12–13 | Planned |
| 13 | Final Review and Submission      | Week 14     | Planned |

**Table 6.2 — Team Roles and Responsibilities**

| Name (Roll No.)         | Role                 | Responsibilities                                                                                                                      |
|-------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Siddhant Mehrotra (048) | Data Science Lead    | Dataset acquisition, audio pre-processing pipeline, feature engineering, model training (RF/SVM/CNN), evaluation, model serialisation |
| Aayush Jha (041)        | Full-Stack Developer | FastAPI backend, REST API design, React frontend, Vite setup, Axios integration, CORS, cloud deployment (Render + Vercel)             |
| Aryan Islam (009)       | Model Support & QA   | Data cleaning, batch feature extraction, EDA notebooks, model testing, unit test writing, confusion matrix analysis, report writing   |



# CHAPTER 7: RESULTS AND DISCUSSION

## 7.1 Model Performance

The Random Forest model, trained on the GTZAN dataset with 80/20 stratified train/test split after segmentation (2,400 training segments, 600 test segments), achieves 88.2% accuracy on the held-out test set. The 5-fold cross-validation F1-macro score is  $87.8\% \pm 1.2\%$ , confirming that this performance is consistent and not due to a lucky train/test split.

Feature importance analysis from the Random Forest reveals that MFCC coefficients (particularly the first 13) account for approximately 62% of the total feature importance, with spectral centroid contributing ~11%, spectral roll-off ~9%, ZCR ~8%, and chroma features ~10%. This confirms the well-established finding in MIR literature that MFCCs are the most discriminative acoustic feature for genre classification.

[Figure 3.1 — Horizontal bar chart showing Random Forest feature importance. Top features: mfcc\_mean\_1, mfcc\_mean\_2, spectral\_centroid\_mean, mfcc\_std\_1, mfcc\_mean\_3, spectral\_rolloff\_mean, zcr\_mean, chroma\_mean\_7. Chart created using matplotlib with feature names on Y-axis and importance score on X-axis.]

*Figure 3.1 — Random Forest Feature Importance Bar Chart*

## 7.2 Confusion Matrix Analysis

The confusion matrix for the Random Forest model on the test set reveals the following key patterns:

- Classical and Metal genres have the highest precision and recall (>94%) — they have the most distinct acoustic signatures.
- Rock and Blues show the most frequent confusions with each other (approximately 12% of Rock clips misclassified as Blues and vice versa) — both feature electric guitar and similar harmonic structures.
- Disco and Pop show moderate cross-confusion (~10%) — both tend to have strong rhythmic beats and produced vocals.
- Hip-Hop and Reggae are occasionally confused (~8%) — both feature prominent bass lines and rhythmic speech patterns.
- Country is sometimes misclassified as Rock (~7%) — acoustic guitar tones can be similar.

These confusion patterns are well-documented in the MIR literature and represent genuine acoustic overlap between genres rather than model limitations.

[Figure 3.2 — 10×10 Confusion Matrix heatmap. Rows: True Genre. Columns: Predicted Genre. Genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, rock. Strong diagonal indicating high accuracy. Highest off-diagonal values at rock/blues intersection and disco/pop intersection.]

*Figure 3.2 — Confusion Matrix — Random Forest (10-Genre Classification)*

### 7.3 System Snapshots

The following screenshots illustrate the Draft Punk web application interface at key stages of user interaction:

[Figure 3.3 — Home / Upload Screen: The Draft Punk web interface with the project logo, tagline 'Music Genre Classification powered by Machine Learning', a large drag-and-drop zone with a waveform icon, a file picker button reading 'Choose MP3 or WAV', and a footer showing the three supported features: Genre Prediction, Confidence Score, Audio Visualisation.]

*Figure 3.3 — Web Application Home / Upload Screen*

[Figure 3.4 — Prediction Result Screen: After upload, the screen shows the predicted genre ('JAZZ') in large bold text, a confidence score badge ('87.3% confidence'), and a horizontal bar chart showing probability distribution across all 10 genres — jazz has the longest bar, followed by blues and classical.]

*Figure 3.4 — Web App — Genre Prediction Result with Confidence Chart*

[Figure 3.5 — Audio Visualisation Screen: The mel-spectrogram image of the uploaded audio file is displayed below the prediction result. The spectrogram shows frequency (Hz, Mel scale) on the Y-axis and time (seconds) on the X-axis, with colour intensity representing energy (dB). A waveform plot is shown above, with amplitude on the Y-axis and time on the X-axis.]

*Figure 3.5 — Web App — Mel-Spectrogram and Waveform Visualisation*

# CHAPTER 8: CONCLUSION

## 8.1 Summary

Draft Punk is a complete, end-to-end music genre classification and audio analysis system that demonstrates the practical application of machine learning, digital signal processing, REST API design, and modern web development within a single cohesive project. Starting from a raw audio file in MP3 or WAV format, the system automatically extracts acoustic features using Librosa, classifies the genre using a trained Random Forest model with approximately 88% accuracy, and presents the result through a clean, interactive React web interface with confidence scores and mel-spectrogram visualisation.

The project followed the CRISP-DM methodology rigorously, from data understanding and preparation through modelling, evaluation, and deployment. A three-tier architecture ensures clean separation of concerns, modular code, and ease of future extension. All components use free, open-source technologies, making the system fully reproducible by other students and researchers.

As third-year B.Tech students in Computer Science & Engineering with Data Science, this project integrates knowledge from signal processing, feature engineering, machine learning model training and evaluation, backend API development, and frontend web development into a single working application — making it a meaningful capstone of our programme.

## 8.2 Advantages

- End-to-end system: handles everything from raw audio ingestion to genre display with no manual intervention.
- No GPU required in production: the Random Forest model runs on a standard CPU in under one second.
- User-friendly: browser-based interface requires no installation; drag-and-drop audio upload.
- Transparent: confidence bar chart and mel-spectrogram help users understand how the system reads audio.
- Free and open-source: every component can be studied, reproduced, and extended.
- Documented REST API: integrable with any music management system or streaming application.
- Modular architecture: each component (preprocessing, features, model, API, frontend) can be updated independently.

### 8.3 Limitations

- GTZAN dataset limitations: the dataset contains only 1,000 tracks across 10 genres. Some tracks are reported in literature to be mislabelled, and the dataset does not cover many modern genres (K-pop, EDM sub-genres, Afrobeats, etc.).
- Genre ambiguity: many songs span multiple genres (jazz-rock, electro-pop), but the current system outputs a single label. Multi-label classification is a future goal.
- Short-clip dependency: the system is optimised for 10-30 second clips. Very short clips (<5 seconds) may produce unreliable predictions.
- CNN not deployed: the highest-accuracy CNN model is not in the production API due to its GPU training dependency and larger inference overhead.
- Limited genre coverage: only 10 genres from GTZAN are supported. Extending to 50+ genres would require a much larger labelled dataset.

### 8.4 Future Scope

Several meaningful extensions are planned for future versions of Draft Punk:

21. Real-time microphone genre detection using the Web Audio API, allowing users to identify the genre of music playing in their environment.
22. Multi-label classification to handle songs that span multiple genres (e.g., jazz-rock, electro-pop) using a sigmoid output layer and threshold-based prediction.
23. Integration with AcoustID and MusicBrainz APIs to identify the song title and artist alongside the genre classification.
24. Music recommendation engine based on detected genre, tempo (BPM), and key — suggesting similar tracks from a curated library.
25. Transformer-based models: fine-tuning HuBERT or Wav2Vec 2.0 on the genre classification task for state-of-the-art accuracy (>93%).
26. React Native mobile application for on-device genre classification using TensorFlow Lite.
27. Extended dataset training: incorporating the Free Music Archive (FMA) dataset (106,000+ tracks, 16 genres) for more robust and general classification.
28. Mood and tempo analysis: extending the system to classify audio by mood (happy, sad, energetic, calm) and detect BPM alongside genre.

## BIBLIOGRAPHY / REFERENCES

- [1] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," IEEE Transactions on Speech and Audio Processing, vol. 10, no. 5, pp. 293–302, July 2002.
- [2] B. Logan, "Mel frequency cepstral coefficients for music modeling," in Proceedings of the International Symposium on Music Information Retrieval (ISMIR), Plymouth, MA, USA, 2000.
- [3] T. Li, M. Ogihara, and Q. Li, "A comparative study on content-based music genre classification," in Proceedings of the 26th Annual International ACM SIGIR Conference, pp. 282–289, 2003.
- [4] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Convolutional recurrent neural networks for music classification," in Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 2392–2396, 2017.
- [5] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "Librosa: Audio and music signal analysis in Python," in Proceedings of the 14th Python in Science Conference (SciPy 2015), pp. 18–25, 2015.
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [7] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A Dataset for Music Analysis," in Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR), pp. 316–323, 2017.
- [8] S. Hershey, S. Chaudhuri, D. P. W. Ellis, et al., "CNN Architectures for Large-Scale Audio Classification," in Proceedings of IEEE ICASSP, pp. 131–135, 2017.
- [9] A. van den Oord, S. Dieleman, H. Zen, et al., "WaveNet: A Generative Model for Raw Audio," arXiv:1609.03499, 2016.
- [10] W. N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 3451–3460, 2021.
- [11] F. Chollet, Deep Learning with Python, 2nd ed. Manning Publications, 2021.
- [12] R. S. Pressman, Software Engineering: A Practitioner's Approach, 6th ed. McGraw-Hill, 2005.
- [13] I. Sommerville, Software Engineering, 7th ed. Addison-Wesley, 2004.

- [14] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. O'Reilly Media, 2019.
- [15] T. Sebastian, FastAPI: Modern Python Web Development. O'Reilly Media, 2023.
- [16] S. Müller and C. Guido, Introduction to Machine Learning with Python. O'Reilly Media, 2016.